INTRODUCTION TO DATABASES · BOILER HOUSE EXAMPLE

Student workbook

A first look at databases and SQL, using one month of data from a boiler house.

What today is about. Not memorising SQL. By the end you should be able to say what a table is, why data gets split across several tables, and how you put it back together — and you will have answered a real engineering question with about six lines of code.

Getting started — two minutes, nothing to install

Everything runs in **Google Colab**, in your browser. You do not need to install Python, and you do not need to download the database — the notebook builds it for you.

Step	What to do
1	Open the link your lecturer gives you. The notebook **boiler_intro.ipynb** opens in Google Colab.
2	**File → Save a copy in Drive.** This is the important one — it gives you your own copy, saved to your Google account, so the answers you type today are still there tomorrow. Work in that copy, not the original.
3	Run the first two cells. Press the ▶ button to the left of a cell, or **Shift + Enter**. The first cell builds the database; the second connects to it and shows you a table.
4	Work down the notebook. Wherever you see `# YOUR TURN`, edit the query and run the cell again.

Things that will happen, and what to do about them

Colab says it disconnected	Run the first two cells again. Nothing is lost.
“no such table: boilers”	You have not run the first two cells yet, or the runtime restarted. Run them.
You want to start a question again	Just retype the query and run the cell. You cannot damage anything.

How you run a query

```
q("SELECT * FROM boilers")
```

You write SQL between the quotes, and `q(...)` hands you back a table you can look at, plot, or carry on working with in Python. That is the whole mechanism.

Prefer not to use Colab? The file **boilerhouse.db** is included in the course pack. Open it with DB Browser for SQLite (free, all platforms) and use the *Execute SQL* tab — every query in this workbook works there unchanged.

The database you will be using

One file, **boilerhouse.db**. It holds one month (April 2026) from a boiler house with three gas-fired boilers. It is deliberately small — you can scroll through any table and check an answer with your own eyes.

Table	What one row is	Columns
boilers	One row per boiler. 3 rows.	boiler_id, code, name, rating_tph, fuel
operators	One row per person. 6 rows.	operator_id, full_name, shift
readings	One row per instrument reading, every 4 hours. 540 rows.	reading_id, boiler_id, ts, steam_tph, stack_temp_c, o2_pct
daily_logs	One row per boiler per day. 90 rows.	log_id, boiler_id, operator_id, log_date, steam_t, fuel_gj
water_tests	One row per weekly water sample. 15 rows.	test_id, boiler_id, test_date, silica_ppm, ph

The two small tables, in full

boiler_id	code	name	rating_tph	fuel
1	B-01	Boiler 1	35.0	Natural gas
2	B-02	Boiler 2	35.0	Natural gas
3	B-03	Boiler 3	30.0	Natural gas

operator_id	full_name	shift
1	Somchai Srisuk	A
2	Naree Wong	A
3	Anan Boonmee	B
4	Pim Chaiyaporn	B
5	Kittipong Intharat	C
6	Wanida Phromma	C

The first few rows of the three big ones

readings

reading_id	boiler_id	ts	steam_tph	stack_temp_c	o2_pct
1	1	2026-04-01 00:00	28.8	125.8	3.17
2	2	2026-04-01 00:00	26.8	176.6	3.17
3	3	2026-04-01 00:00	25.7	128.6	3.36
4	1	2026-04-01 04:00	29.1	125.6	3.23
5	2	2026-04-01 04:00	26.0	179.3	3.28

daily_logs

log_id	boiler_id	operator_id	log_date	steam_t	fuel_gj
1	1	2	2026-04-01	680.9	2072.3
2	2	3	2026-04-01	633.3	1979.0
3	3	4	2026-04-01	573.0	1753.4
4	1	3	2026-04-02	703.9	2142.4
5	2	4	2026-04-02	647.1	2023.2

water_tests

test_id	boiler_id	test_date	silica_ppm	ph
1	1	2026-04-01	13.1	10.97
2	2	2026-04-01	9.8	11.01
3	3	2026-04-01	7.5	10.73
4	1	2026-04-08	7.4	11.09
5	2	2026-04-08	8.9	11.0

Notice how the tables connect. A row in **readings** does not say “Boiler 2” — it says **boiler_id = 2**, and you look 2 up in the **boilers** table to find out what it is. That single idea, storing a fact once and pointing at it, is what makes a database a database rather than a spreadsheet.

Everything you need, on one page

Write this	To do this
`SELECT * FROM boilers;`	Every column, every row
`SELECT name, fuel FROM boilers;`	Just these columns
`WHERE rating_tph > 30`	Only rows matching a condition
`WHERE boiler_id = 2 AND o2_pct < 3`	Two conditions at once
`WHERE ts LIKE '2026-04-01%'`	Text starting with something
`ORDER BY stack_temp_c DESC`	Sort, biggest first (ASC = smallest first)
`LIMIT 10`	Only the first 10 rows
`COUNT(*)`	How many rows
`AVG(x), MIN(x), MAX(x), SUM(x)`	Average, smallest, biggest, total
`ROUND(x, 1)`	Round to 1 decimal place
`GROUP BY boiler_id`	One answer per boiler instead of one overall
`AS mean_stack_c`	Give the answer column a readable name
`JOIN boilers b ON b.boiler_id = r.boiler_id`	Look up the matching row in another table

The shape of every query you will write today

```
SELECT  which columns you want
FROM    which table
JOIN    another table  ON  how they match     (only when you need it)
WHERE   which rows you want                   (optional)
GROUP BY what to summarise by                 (optional)
ORDER BY how to sort                          (optional)
LIMIT   how many rows to show                 (optional)
```

Only SELECT and FROM are compulsory. Everything else you add when you need it, and always in that order.

Exercise 1 · Looking at the data

25 minutes · SELECT · WHERE · ORDER BY · LIMIT · COUNT

Get a feel for what is in the database, and learn the three things every query does: choose columns, choose rows, and put them in order.

1.1

Show the whole **boilers** table.

Your query and answer:

1.2

Show the **name** and **rating_tph** of the boilers rated above 30 t/h.

Your query and answer:

1.3

How many rows are in the **readings** table?

Your query and answer:

1.4

Show the five hottest stack temperature readings — all columns, hottest first.

Your query and answer:

1.5

Show every reading for **boiler 2** taken on **1 April 2026**.

Your query and answer:

Exercise 2 · Summarising the data

25 minutes · AVG · MIN · MAX · GROUP BY

Turn many rows into one number, then into one number per boiler. This is where a database starts to save real time.

2.1

What is the average steam flow across every reading?

Your query and answer:

2.2

What is the average **stack temperature** for each boiler? Look at the answer — is anything odd?

Your query and answer:

2.3

For each boiler, show the lowest and highest stack temperature.

Your query and answer:

2.4

How many readings does each boiler have?

Your query and answer:

2.5

From **daily_logs**, show the total steam and total gas for each boiler for the month.

Your query and answer:

Exercise 3 · Joining two tables

30 minutes · JOIN … ON · JOIN with GROUP BY

See why the data is split across tables in the first place, and put it back together with a JOIN.

3.1

The readings table only says **boiler_id = 2**. Show the first 10 readings with the boiler's **name** instead of its number.

Your query and answer:

3.2

Show the first 10 daily logs with the boiler **code** and the **operator's name**.

Your query and answer:

3.3

Average stack temperature per boiler again — but this time show the boiler **code**, not the id.

Your query and answer:

3.4

For each boiler, work out the **gas used per tonne of steam** (total gas ÷ total steam) and show it with the boiler code. Which boiler is the worst, and by how much?

Your query and answer:

3.5

No query for this one. Put 2.2 and 3.4 side by side. Why do you think B-02 uses more gas? Write one sentence.

Your query and answer:

If you get stuck

What you see	What it usually means
no such column: B-02	You used double quotes or none at all. Text goes in single quotes: `code = 'B-02'`.
no such table: Boilers	Check the spelling. The tables are boilers, operators, readings, daily_logs, water_tests.
An empty result	Your WHERE is too strict, or a value is spelled differently from what you typed. Take the WHERE line off and look at the raw data first.
near "FROM": syntax error	Almost always a missing comma between column names, or an extra one before FROM.
The answer looks wrong	Run the query without GROUP BY or without the JOIN and look at the rows. Build a query up one line at a time.

Three things worth remembering

- **A table is a list of one kind of thing.** One row = one boiler, or one reading, or one day. If a row is trying to be two things at once, it should be two tables.
- **Store a fact once.** The boiler's rating lives in one row of one table. Everything else points at it. That is why changing it is safe.
- **The computer counts, you interpret.** SQL told us B-02 runs 30 °C hotter and burns 4 % more gas. Only an engineer can say that means the tubes need cleaning.